



Graduate Diploma in Software Engineering

ITS 1018 – Software Architectures & Design Patterns II

Final Examination Assignment

Smart Parking Management System - Microservice-Based Application

Total Marks: 100

Smart Parking Management System (SPMS)

Microservice-Based Application

Business Scenario

Background

Urban environments are facing an ongoing crisis of space utilization, primarily due to the increasing population density and vehicle ownership. Parking has become a major daily challenge in cities where drivers spend a significant amount of time searching for available spots. This problem not only affects individuals but also burdens public infrastructure and environmental conditions. The following issues are commonly observed:

- Daily traffic congestion and long vehicle queues
- Fuel wastage and driver frustration
- Rise in greenhouse gas emissions
- Reduced urban mobility and safety
- Loss of business productivity due to limited access to parking

Municipal authorities and private operators have tried manual solutions and static systems, but none are efficient, scalable, or capable of adapting to real-time demand.

Proposed Solution: Smart Parking Management System (SPMS)

To solve these challenges, we propose the **Smart Parking Management System (SPMS)** - a cloud-native, **microservice-based platform**. The system is designed to offer real-time management and monitoring of parking spaces while allowing users to easily locate, reserve, and pay for parking. It will serve three **key stakeholders**:

- End Users (Drivers)
- Parking Owners (Space providers)
- Payment Systems (Mock gateways)

The system's features include:

- Dynamic space allocation and availability tracking
- Real-time updates via manual or IoT integrations
- Digital transactions and booking receipts
- Scalable, decoupled architecture using microservices
- Modular design for rapid deployment and maintainability

By aligning with smart city goals, SPMS promotes sustainable urban mobility, reduced congestion, and digital transformation of city services.

Business Objectives:

- Allow users to search and reserve parking in real time.
- Enable parking owners to monitor and manage their spaces dynamically.
- Track parking space usage per city, zone, and owner.
- Simulate vehicle entry and exit.
- Handle payments securely and generate digital receipts.
- Provide historical logs for users and administrators.
- Reduce traffic congestion and increase parking efficiency.

Key Technologies & Tools

Technology	Purpose
Spring Boot	Framework for building production-ready microservices with minimal configuration. Core technology for all microservices.
Spring Cloud Eureka	Service registry for dynamic registration and discovery of microservices.
Spring Cloud Config	Centralized configuration management for distributed microservices.
Spring Cloud Gateway	API Gateway serving as a single-entry point to route client requests to backend microservices.
Spring Web	Provides RESTful API development capabilities within Spring Boot microservices.
Node.js with Express	Optional lightweight framework for implementing services if desired.
Python Flask	Optional lightweight microframework for building REST APIs if desired.
Postman	Tool for testing and validating API endpoints across the microservices.

Note : *The entire microservice architecture must primarily use Spring Boot. Node.js or Python may be used selectively where appropriate, but the system should primarily run on Spring Boot.*

Steps and Microservices Architecture Explanation

Service Registry and Discovery (Eureka):

- **Purpose** : Eureka enables microservices to register themselves and discover other services dynamically.
- **Implementation** : Each microservice registers with Eureka upon startup, facilitating communication without hard coded dependencies.

Configuration Server (Spring Cloud Config):

- **Purpose** : Centralized configuration management.
- **Implementation** : Microservices fetch configurations from the Config Server, allowing updates without redeployment.

API Gateway (Spring Cloud Gateway):

- **Purpose** : Serve as a single-entry point for client requests.
- **Implementation** : Routes requests to appropriate microservices based on URI and HTTP method, leveraging Eureka for dynamic service discovery.

Microservices and Business Logic:

Note: *You can use any of the following technologies for implementing these services based on suitability: Spring Boot, Node.js (Express), or Python (Flask)*

Parking Space Service (Spring Boot / Python / Node.js)

Business Logic: *Manages available parking spots.*

- ❖ *List and manage parking spaces.*
- ❖ *Reserve and release parking spaces.*
- ❖ *Update status as occupied or available.*
- ❖ *Filter by location, availability, etc.*
- ❖ *Update space status manually or simulated IoT.*

Vehicle Service (Spring Boot / Python / Node.js)

Business Logic: *Handles vehicle operations.*

- ❖ *Register, update and retrieve vehicle details.*
- ❖ *Link vehicles to users.*
- ❖ *Simulate vehicle entry / exit tracking.*

User Service (Spring Boot / Python / Node.js)

Business Logic: *Manages user and owner information.*

- ❖ *Register and authenticate users.*
- ❖ *View and update user profiles.*
- ❖ *Access booking history and logs (in-memory or persistent)..*

Payment Service (Spring Boot / Python / Node.js)

Business Logic: *Handles internal payment transactions for parking fees.*

- ❖ *Validate mock card/payment data.*
- ❖ *Simulate transaction flow and status.*
- ❖ *Generate digital receipts.*

Testing Approach

- Use **Postman** to test all API endpoints.
 - Focus exclusively on backend functionality (excluding UI development).
 - Validate functionality across all microservices.
 - Check error handling and edge case behavior thoroughly.
-

Submission Guidelines

1. GitHub Repository:

- Push your entire project to a GitHub repository.
- Organize your project with a proper folder structure.
- Include a `README.md` file in the root directory.
- The `README.md` should contain references (links) to:
 - Your Postman collection file.
 - Eureka dashboard screenshot

2. Postman Collection:

- Export your Postman collection as a `.json` file.
- Place this file in the root directory of your project.
- Reference it in your `README.md` using this format:

```
[Postman Collection](./postman_collection.json)
```

3. Eureka Dashboard Screenshot:

- Take a screenshot of the Eureka dashboard showing all your registered services.
- Save the screenshot file inside the folder:
`docs/screenshots/`
- Name the screenshot file descriptively, e.g.,
`eureka_dashboard.png`
- Reference the screenshot in your `README.md` using markdown:

```
![Eureka Dashboard](./docs/screenshots/eureka_dashboard.png)
```

Example Markdown for README:

```
# Project Name
## Resources
- [Postman Collection](./postman_collection.json)
- ![Eureka Dashboard](./docs/screenshots/eureka_dashboard.png)
```